

9. Malicious Software

Sicurezza dell'Informazione

Cos'è un malware

Un **portmanteau** (fusione) di “**MALicious softWARE**”

Significato: codice scritto intenzionalmente per violare una politica di sicurezza.

Esistono diverse “categorie” o “caratteristiche” in base a come si **diffondono/propagano** e se hanno o meno **bisogno dell'intervento umano**:

- **Virus:** codice che si auto-propaga infettando eseguibili o altri file (ma anche documenti con macro, boot loader). Non sono programmi autonomi (cioè non sono eseguibili indipendenti). **La diffusione avviene localmente.**
- **Worm:** programmi che si auto-propagano, anche a distanza, spesso sfruttando vulnerabilità dell'host o tramite social engineering (ad esempio, worm diffusi via e-mail). **La diffusione è remota.**
- **Trojan horses:** programmi apparentemente benigni che nascondono una funzionalità malevola e permettono il controllo remoto.

Una breve storia dei malware... Virus

1971 – **Creeper** è il primo programma auto-replicante su PDP-10

Non era malevolo. Era un esperimento per dimostrare se un programma potesse spostarsi tra diversi computer sulla rete ARPANET.

Creeper utilizzava i protocolli di ARPANET (non il moderno TCP/IP!) per:

- Connettersi a un altro PDP-10.
- Trasferire una copia di sé stesso.
- Visualizzare il messaggio:



I'M THE CREEPER. CATCH ME IF YOU CAN!

È importante notare che Creeper si spostava anziché replicarsi indefinitamente, il che significa che abbandonava il sistema precedente quando si copiavano su uno nuovo — più simile a un “agente mobile” che a un virus.

Una breve storia dei malware... Virus

1981 – **Elk Cloner** è il primo virus a diffondersi ampiamente su Apple II tramite floppy disk

Scritto da Richard Skrenta, allora quindicenne.

Non era pensato per essere malevolo, ma come uno scherzo rivolto ai compagni di scuola.

Si diffondeva sfruttando i floppy disk di avvio:

- Quando un Apple II veniva avviato da un disco infetto,
- Elk Cloner si caricava in memoria.
- Successivamente infettava qualsiasi altro floppy inserito.

Ogni 50° avvio, mostrava una poesia chiamata "Elk Cloner: The Program with a Personality".



```
Elk Cloner:  
The program with a personality  
  
It will get on all your disks  
It will infiltrate your chips  
Yes it's Cloner!  
  
It will stick to you like glue  
It will modify ram too  
Send in the Cloner!
```

Teoria dei virus informatici

Fred Cohen (1983) teorizzò l'esistenza dei virus e produsse i primi esempi di virus informatici.

Da un punto di vista della teoria dell'informatica, rappresentano un concetto interessante di codice auto-modificante e auto-propagante.

Ben presto furono compresi anche le sfide e i limiti della sicurezza.

È impossibile costruire un rilevatore di virus perfetto (cioè un rilevatore perfetto della propagazione).

Sia P un programma di rilevamento perfetto.

Possiamo costruire un virus V che risponde in modo contrario a P :

- se $P(V) = \text{True} \rightarrow V$ si arresta (non si propaga) $\rightarrow$ quindi V non è un virus
- se $P(V) = \text{False} \rightarrow V$ si propaga $\rightarrow$ quindi V è un virus

Ciclo di vita del codice malevolo

Riprodursi, infettare, rimanere nascosto, eseguire il payload

Fase di riproduzione e infezione:

- Bilanciare la capacità d'infezione con la possibilità di essere rilevati.
- Necessità di identificare un vettore di propagazione adeguato (che può essere social engineering, exploit di vulnerabilità...).
- Necessità di infettare file (solo per i virus) e propagarsi ad altre macchine.
- Nota: la maggior parte del malware moderno non si auto-propaga affatto (la maggior parte di bot e trojan non sono worm).

Varietà di tecniche per rimanere nascosti.

Payload: talvolta dannoso... altre volte no, semplicemente fastidioso.

- Ping-pong: https://www.youtube.com/watch?v=v1-jNkzBx_M
- Yerg: <https://youtu.be/96600sXubxw>
- Cascade: <https://www.youtube.com/watch?v=gN7JKH0VnJ0>
- YAAI: <https://youtu.be/LSgk7ctw1HY?t=54>

Tecniche di infezione

Virus di boot

- Infettano il Master Boot Record (MBR) del disco rigido (Hard Disk, SSD) per caricare il malware prima del sistema operativo.

File infector (virus che infettano i file)

- Simple overwrite virus: sovrascrive il programma originale (lo danneggia). Tendenzialmente non utilizzato in quanto ci si accorge che il programma non funziona più.
- Parasitic virus:
 - aggiunge codice al file e modifica il punto di ingresso del programma per far eseguire il proprio codice.
- (Multi) cavity virus:
 - inietta codice in aree inutilizzate all'interno del file eseguibile.

Una breve storia dei malware... Virus

1987 – Christmas Worm (CHRISTMA EXEC) sui mainframe IBM

Primo grande mass-mailing worm su sistemi IBM VM/CMS.

Mostrava un'animazione ASCII natalizia e si auto-inviava ai contatti dell'utente.

Diffusione massiva: fino a 500.000 repliche/ora, causando congestione delle reti accademiche.

Dimostrò che anche sistemi centralizzati potevano essere paralizzati da codice auto-propagante.



Una breve storia dei malware... Virus

1988 – Morris Internet Worm

Primo worm a diffondersi su larga scala su Internet.

Sfruttava vulnerabilità in sendmail, finger, rsh e password deboli.

Una scelta progettuale (reinfezione casuale) causò sovraccarico e blocco di migliaia di sistemi (tra 6.000 e 10.000 macchine).

Evento storico: portò alla creazione del CERT (Computer Emergency Response Team).

Considerato il primo grande incidente di sicurezza della storia di Internet.

Una breve storia dei malware... Virus

1998 – Back Orifice

Rilasciato dal *Cult of the Dead Cow* per dimostrare la mancanza di sicurezza nei sistemi Windows dell'epoca.

Trojan che permetteva controllo remoto completo del sistema infetto.

Facilissimo da usare e diffondere, popolare nei canali IRC.

Rese evidente il problema delle backdoor e della **scarsa protezione dei sistemi consumer anni '90**.



Persistenza e controllo remoto

Una **backdoor** (porta sul retro) è un meccanismo che permette l'accesso nascosto a un sistema, aggirando le normali misure di autenticazione o sicurezza.

Può essere introdotta da:

- software malevolo (trojan, RAT),
- vulnerabilità intenzionali o accidentali,
- configurazioni errate,
- insider malintenzionati.

Obiettivo tipico:

- controllo remoto,
- raccolta di informazioni,
- installazione di ulteriore malware,
- persistenza nel sistema.

Le backdoor diventano spesso parte di una botnet, permettendo il **controllo coordinato di molte macchine infette**.

Da non confondere con gli exploit: una backdoor rimane nel sistema e permette accesso continuo nel tempo.

Una breve storia dei malware... Virus

1999 – Melissa Virus

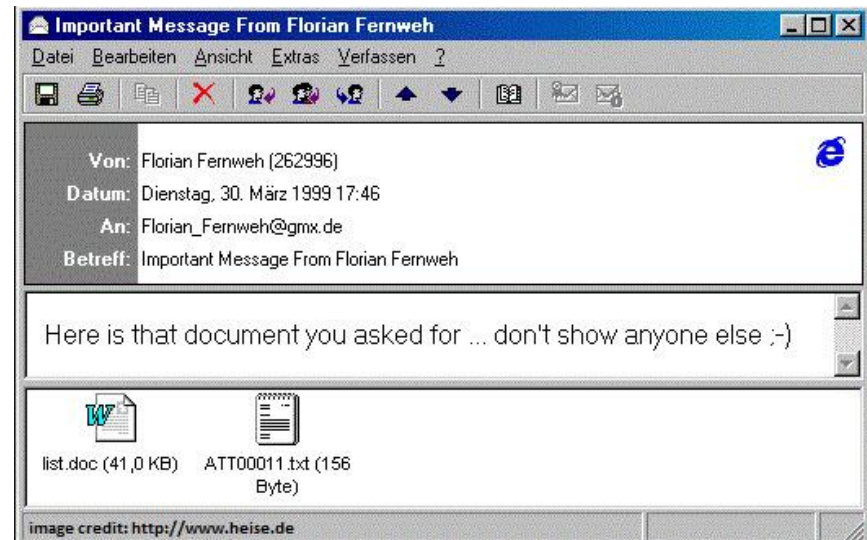
Macro-virus per Microsoft Word diffuso tramite email.

Si auto-inviava ai primi 50 contatti della rubrica dell'utente, causando traffico massivo sulle reti corporate.

All'utente bastava aprire un allegato per infettarsi.

Primo grande caso di email-borne macro virus su scala globale (il concetto di macro virus risale al 1995).

Costrinse molte aziende, incluse Microsoft e FBI, a intervenire rapidamente
→ forte impatto mediatico.



“Macro” Virus

I file di dati erano tradizionalmente considerati sicuri dai virus.

Le funzionalità di macro aggiungono codice ai file di dati.

Esempio: le macro di un foglio di calcolo possono:

- modificare un foglio di calcolo,
- modificare altri fogli di calcolo,
- accedere alla rubrica,
- inviare email,
- ...

Difficile da rimuovere.

...che si trasforma in una storia di worm

1999 – I primi attacchi DDoS tramite macchine “trojanizzate” (zombies)

Per la prima volta compaiono attacchi DDoS distribuiti su larga scala condotti tramite PC compromessi (detti zombies).

Le macchine infette erano controllate tramite trojan/backdoor, costituendo le prime forme rudimentali di botnet.

Gli attacchi più noti coinvolsero flood di traffico verso bersagli ad alta visibilità (università, provider, siti istituzionali).

Segnarono l'inizio del passaggio da malware “dimostrativo” a strumento reale di attacco organizzato.

Premessa fondamentale per gli attacchi DDoS di inizio anni 2000 (MafiaBoy, Trinoo, TFN).

...che si trasforma in una storia di worm

1999 – I rootkit del kernel diventano pubblici

Pubblicati i primi kernel rootkit moderni per Linux, tra cui **Knark**.

Segna l'inizio della sofisticazione del malware orientato alla persistenza e alla stealth mode a livello kernel (sistema operativo).

Questi rootkit resero evidente che il malware poteva operare sotto il sistema operativo, diventando invisibile agli strumenti tradizionali.

...che si trasforma in una storia di worm

2000 – ILOVEYOU (email worm su larga scala) – Social Engineering

Worm di massa distribuito tramite email con oggetto “ILOVEYOU”.

Allegato mascherato: LOVE-LETTER-FOR-YOU.TXT.vbs → script eseguito dall'utente pensando fosse un file di testo.

Esempio emblematico di social engineering: la curiosità dell'utente era il vettore di infezione.

Il worm si auto-inviava ai contatti di Outlook, diffondendosi rapidamente in tutto il mondo.

Danni enormi: interruzioni di servizio, reti aziendali bloccate, milioni di sistemi coinvolti.

...che si trasforma in una storia di worm

2001 – Code Red (worm su larga scala basato su exploit)

Worm che sfruttava una vulnerabilità in Microsoft IIS (**buffer overflow** in idq.dll).

Nessuna interazione dell'utente: infezione completamente exploit-based e autonoma.

Si propagava effettuando scanning di indirizzi IP casuali alla ricerca di server vulnerabili.

Divenne una delle prime grandi epidemie Internet: centinaia di migliaia di sistemi compromessi.

Effetti: deturpazione di siti web (es., codice HTML modificato con slogan) e attacchi DDoS (es. contro il sito della Casa Bianca).

Segnò l'inizio dell'era dei worm veloci e automatizzati basati su vulnerabilità remote.

...che si trasforma in una storia di worm

2003 – SQL Slammer (worm estremamente rapido tramite UDP)

Worm di dimensioni minuscole (~376 byte!) che sfruttava una vulnerabilità in Microsoft SQL Server / MSDE.

Si diffondeva tramite UDP (porta 1434), quindi senza handshake → propagazione fulminea.

Infettò la maggior parte delle vittime in meno di 10 minuti; picco globale in circa 3 minuti.

Causò congestione massiva delle reti, blackout di router, rallentamenti di banche, aeroporti e servizi cruciali.

È uno dei casi più studiati di worm “flash”, esempio perfetto di propagazione epidemica esponenziale.

Worm moderni: scanner di massa

Lo schema di base è lo stesso: infettare un computer e cercare nuovi bersagli.

Diffusione più rapida (minuti) e su larga scala (centinaia di migliaia di host).

Scanning:

- **Selezione di indirizzi casuali.**
- **Local preference:** più scansioni verso la rete locale.
- **Permutation scanning:** suddivisione dello spazio di indirizzamento IP.
- **Hit list scanning:** utilizzo di una lista preliminare di host vulnerabili.
- **Warhol worm:** combinazione di hit list + permutation scanning.

Paper "How to Own the Internet in your spare time",

<http://www.icir.org/vern/papers/cdc-usenix-sec02/>

Silence on the Wires

Pensavamo tutti che Internet sarebbe diventata sempre più “piena di worm”.

Dal 2004, però, “silenzio sulle linee”: nessuna nuova grande epidemia di worm.

Le vulnerabilità sfruttabili c'erano, e più di una volta ci siamo persino preparati al peggio :-)

Dove sono finiti gli autori dei worm?

Domande correlate...

Perché nessun worm ha mai preso di mira l'infrastruttura di Internet?

Un worm tradizionale per la propagazione + un payload specializzato per danneggiare l'infrastruttura.

Le finestre di opportunità ci sono state:

- **Giugno 2003:** MS03-026, vulnerabilità RPC-DCOM (Blaster) + Blocco delle interfacce Cisco IOS tramite pacchetti IPv4
- **Aprile 2004:** MS04-011, vulnerabilità LSASS (Sasser) + reset TCP su diversi prodotti Cisco IOS

Quindi perché **/bin/laden** non ha colpito?

Domande correlate...

Perché nessun worm ha mai preso di mira l'infrastruttura di Internet?

Un worm tradizionale per la propagazione + un payload specializzato per danneggiare l'infrastruttura.

Le finestre di opportunità ci sono state:

- **Giugno 2003:** MS03-026, vulnerabilità RPC-DCOM (Blaster) + Blocco delle interfacce Cisco IOS tramite pacchetti IPv4
- **Aprile 2004:** MS04-011, vulnerabilità LSASS (Sasser) + reset TCP su diversi prodotti Cisco IOS

Quindi perché **/bin/laden** non ha colpito?

Risposta: mancanza di motivazione — gli attaccanti hanno bisogno che l'infrastruttura rimanga attiva per poter fare ciò che devono fare.

... ma anche ...

Gli attaccanti oggi sono interessati a **monetizzare il loro malware**.

Monetizzazione diretta (ad esempio, abuso di carte di credito, connessioni a numeri a tariffazione speciale).

Monetizzazione indiretta

- Raccolta di informazioni
- Abuso delle risorse di calcolo
- Affitto o vendita di infrastrutture di botnet

Tutto ciò ha creato una crescente **underground economy**.

L'ecosistema Cybercrime

Gruppi organizzati di cybercriminali

Diverse “attività”:

- sviluppo e acquisizione di exploit
- infezione di siti
- monitoraggio delle vittime
- vendita di “exploit kit”
- ...e offrono persino supporto ai loro clienti.

Approfondimento

“Manufacturing Compromise: The Emergence of Exploit-as-a-Service”

<http://cseweb.ucsd.edu/~voelker/pubs/eaas-ccs12.pdf>

...che si trasforma in una storia di worm

Dal 2004 in poi emergono malware progettati principalmente per costruire reti di macchine compromesse (botnet).

Obiettivi:

- controllo remoto su larga scala,
- monetizzazione (spam, phishing, furto credenziali),
- affitto/vendita dell'infrastruttura,
- operazioni di cyber-crimine organizzato.

Esempi celebri:

- Storm Worm – mass-mailing + P2P, difficile da smantellare
- Torpig / Mebroot – furto credenziali bancarie, infrastruttura sofisticata
- Koobface – propagazione via social network
- Conficker – exploit-based, milioni di sistemi infetti
- Stuxnet – malware industriale altamente mirato, dimostrazione dell'uso statale del malware

Segna il passaggio dal malware “dimostrativo” ai sistemi criminali su larga scala, spesso dotati di persistenza, stealth e comandi remoti complessi.

L'ascesa dei Bot

Abuso di bot IRC (IRCwars):

- **IRCwars**: uno dei primi attacchi DDoS documentati.
- I canali IRC (Internet Relay Chat) venivano utilizzati come meccanismo di coordinamento dei primi "bot".
- Molti utenti malevoli riuscivano a prendere controllo dei bot attraverso IRC e li usavano per lanciare attacchi DDoS rudimentali.
- Questi conflitti tra gruppi IRC sono tra i primissimi casi documentati di attacchi DDoS organizzati.

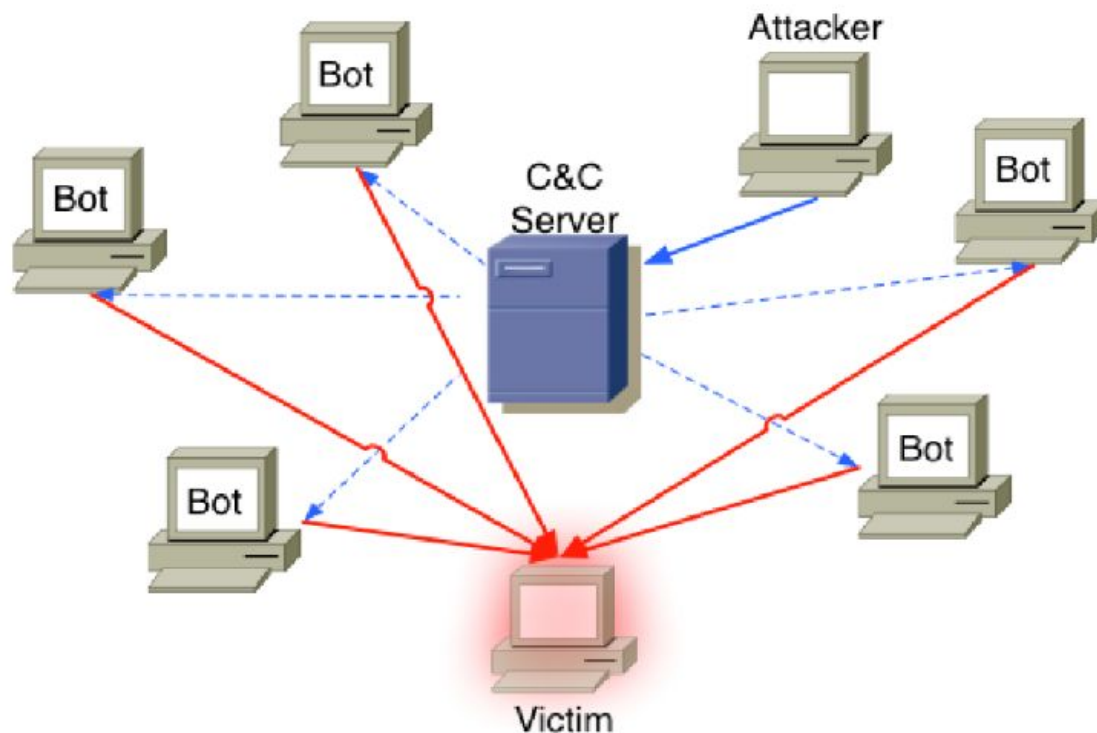
1999: **trinoo**, "strumento di attacco DDoS"

- originariamente girava su Solaris (poi portato su Windows).
- la configurazione della botnet era perlopiù manuale.
- Agosto 1999: attacco DDoS contro un server dell'Università del Minnesota usando almeno 227 bot.
(<http://staff.washington.edu/dittrich/misc/trinoo.analysis>)

Anni 2000: attacchi DDoS contro siti di grande visibilità (Amazon, CNN, eBay) ottennero enorme copertura mediatica.

Botnet

Una botnet è una rete composta da diversi bot malevoli che sono controllati da un comandante, comunemente chiamato botmaster (o botherder).



PhatBot: comandi d'esempio

- Raccogliere indirizzi email dall'host
- Registrare tutti i tasti premuti (keylogging)
- Sniffare il traffico di rete
- Catturare screenshot
- Avviare un server HTTP che permette di sfogliare il disco C:
- Terminare una lista predefinita di processi
 - Programmi antivirus
 - Malware concorrenti
- Rubare le product key di Windows
 - Rubare anche le key di giochi popolari
- Proxy SOCKS
 - Imposta un proxy da usare come "stepping stone" per inviare SPAM
- Scaricare un file da un URL
- Eseguire un comando di shell
- Aggiornamento
 - Permette di modificare i comandi disponibili!

...che si trasforma in una storia di worm

Scareware:

- Finti antivirus o tool di pulizia che mostrano allarmi falsi.
- Inducono l'utente a pagare per "riparare" problemi inesistenti.

Ransomware:

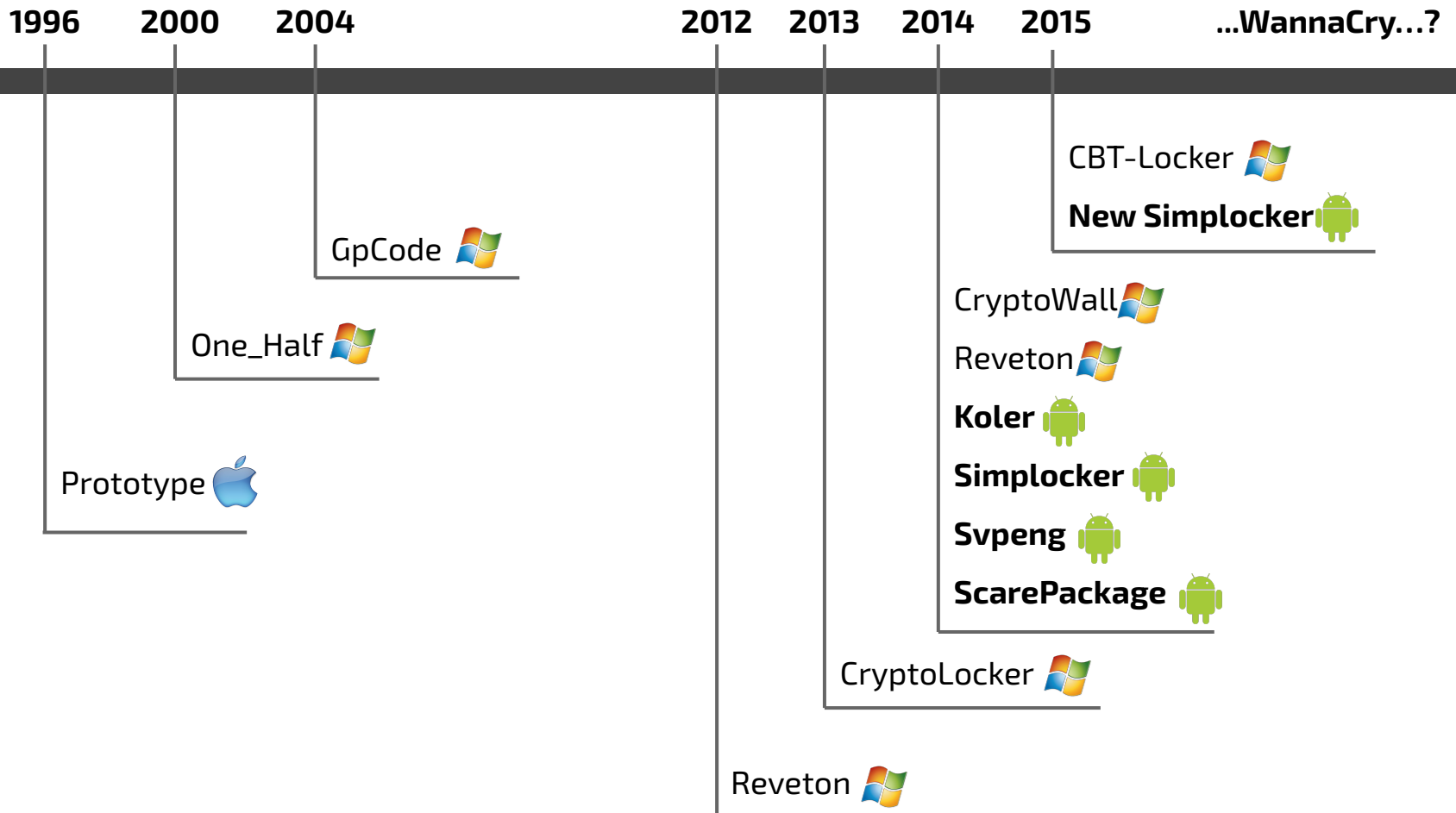
- Cifra i file della vittima e richiede un riscatto per decriptarli.
- Diventa una delle minacce più redditizie e diffuse a partire dagli anni 2010.
- Esempi: CryptoLocker, WannaCry, NotPetya.

Malware sponsorizzato da stati (APT):

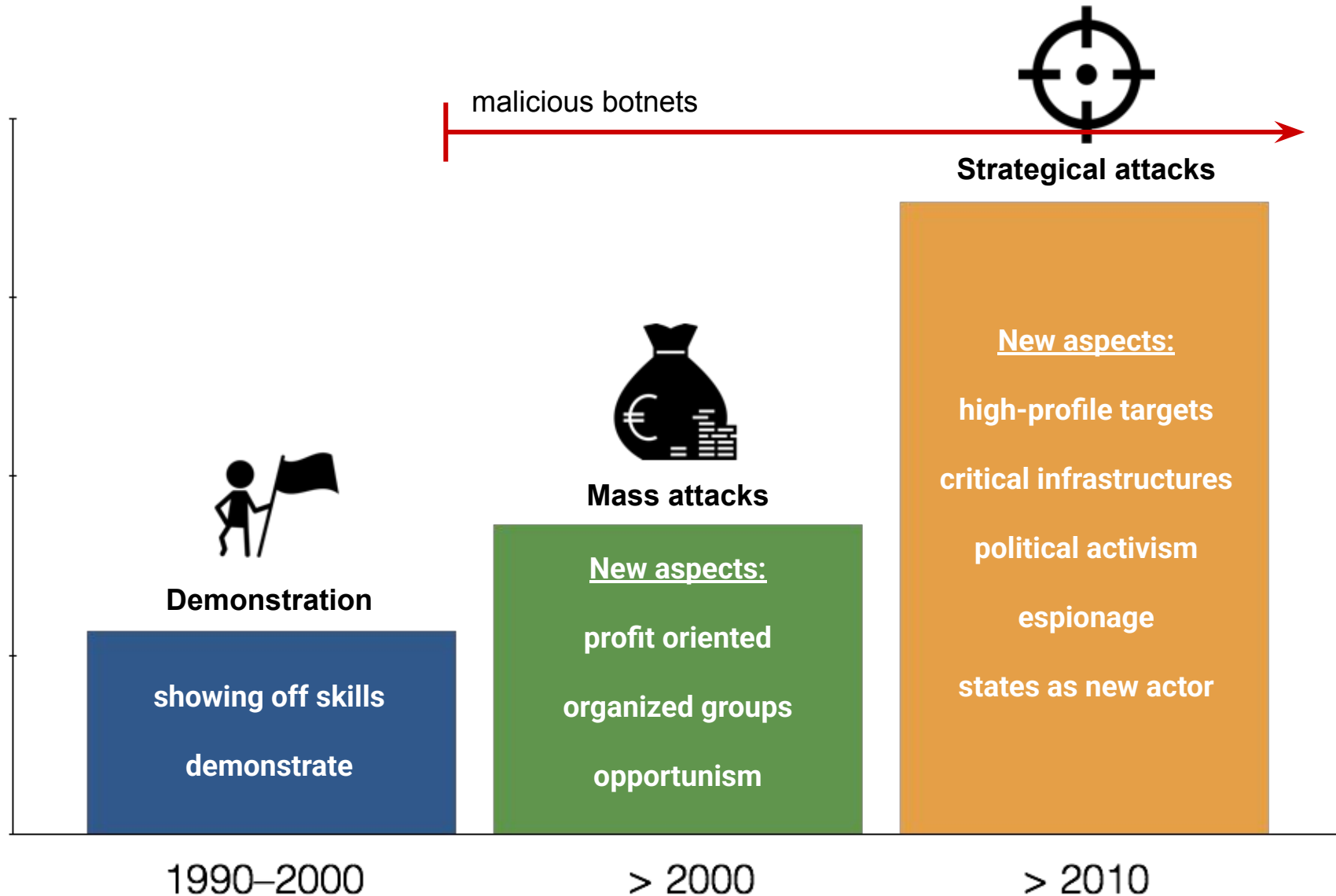
- Progettato per operazioni strategiche: spionaggio, sabotaggio, attacchi mirati.
- Altamente sofisticato e spesso rivolto a infrastrutture industriali o governative.
- Esempi: Stuxnet, Duqu, Flame, Regin.

Segna l'evoluzione del malware da semplice disturbo a vera arma economica, criminale e geopolitica.

Storia dei Ransomware



30 anni di software malevolo



Difendersi contro i malware

Patch

La maggior parte dei worm sfrutta vulnerabilità note.

Rilevare il malware

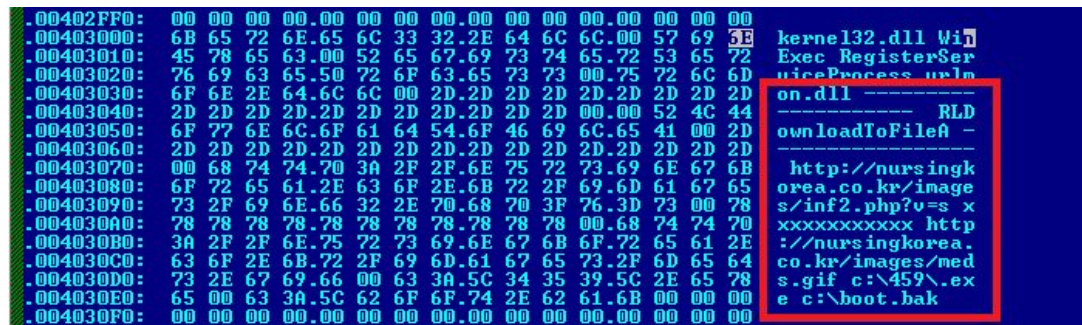
- **Firme** (cosa è il malware)
 - Schemi o sequenze di codice unici e riconoscibili. Devono essere create automaticamente: i worm operano troppo velocemente perché una risposta manuale sia efficace.
- **Comportamenti** (cosa fa il malware)
 - Azioni note compiute su un sistema (es. furto di dati, modifica di file, spionaggio sull'utente, propagazione, interruzione delle operazioni).
- **Rilevamento di intrusioni o anomalie** (effetti sul sistema/rete infetti)
 - Identificare diffusione rapida, attività sospette.
 - Può essere un punto di partenza per la generazione automatica di firme.

Antivirus e Anti-malware

Software progettato per rilevare, bloccare e rimuovere software malevolo (malware) da computer e reti.

Strategia di base: rilevamento tramite firme

- un database di firme a livello di byte o di istruzioni che corrispondono a malware conosciuti



```
.00402FF0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
.00403000: 6B 65 72 6E 65 6C 33 32 2E 64 6C 6C 00 57 69 5E kernel32.dll Wi
.00403010: 45 78 65 63 00 52 65 67 69 73 74 65 72 53 65 72 Exec RegisterSer
.00403020: 76 69 63 65 50 72 6F 63 65 73 73 00 75 72 6C 6D uiceProcess_urlm
.00403030: 6F 6E 2E 64 6C 6C 00 2D 2D 2D 2D 2D 2D 2D 2D 2D on.dll -----
.00403040: 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D ----- RLD
.00403050: 6F 77 6E 6C 6F 61 64 54 6F 46 69 6C 65 41 00 2D ownloadToFileA -
.00403060: 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D
.00403070: 00 68 74 74 70 3A 2F 2F 6E 75 72 73 69 6E 67 6B
.00403080: 6F 72 65 61 2E 63 6F 2E 6B 72 2F 69 6D 61 67 65
.00403090: 73 2F 69 6E 66 32 2E 70 68 70 3F 76 3D 73 00 78
.004030A0: 78 78 78 78 78 78 78 78 78 78 00 68 74 74 70
.004030B0: 3A 2F 2F 6E 75 72 73 69 6E 67 6B 6F 72 65 61 2E
.004030C0: 63 6F 2E 6B 72 2F 69 6D 61 67 65 73 2F 6D 65 64
.004030D0: 73 2E 67 69 66 00 63 3A 5C 34 35 39 5C 2E 65 78
.004030E0: 65 00 63 3A 5C 62 6F 6F 74 2E 62 61 6B 00 00 00
.004030F0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
.00403100: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

Virus Name	String Pattern (Signature)
Accom.1280	89C3 B440 8A2E 2004 8A0E 2104 BA00 05CD 21E8 D500 BF50 04CD
Die.448	B440 B9E8 0133 D2CD 2172 1126 8955 15B4 40B9 0500 BA5A 01CD
Xany.979	8B96 0906 B000 E85C FF8B D5B9 D303 E864 FFC6 8602 0401 F8C3

- possono essere utilizzati wildcard, ed espressioni regolari sono comuni

Antivirus e Anti-malware

Software progettato per rilevare, bloccare e rimuovere software malevolo (malware) da computer e reti.

Strategia di base: rilevamento tramite firme

- database di firme a livello di byte o di istruzioni che corrispondono a malware conosciuti
- è possibile utilizzare wildcard; le espressioni regolari sono comuni

Euristiche (verifica di segni di infezione)

- l'esecuzione del codice non inizia dove dovrebbe
- nome sospetto della sezione di codice

Rilevamento comportamentale

- rilevare i segni (il comportamento) di malware conosciuti
- rilevare "comportamenti/azioni comuni" tipici del malware

Analisi e identificazione dei malware

Workflow ex-post:

1. Un eseguibile sospetto viene segnalato da “qualcuno”.
Analizzato automaticamente e manualmente.
2. Vengono sviluppate firme antivirus e comportamenti/euristiche.

Analisi statica

- Analisi del codice binario senza eseguirlo.
- Tecniche: disassemblaggio, decompilazione, estrazione di stringhe, analisi del flusso di controllo.
- Estrae firme / euristiche.

Analisi dinamica

- Comporta l'esecuzione del malware in un ambiente controllato per monitorarne il comportamento.
- Tecniche: sandboxing, hooking, tracing, monitoraggio delle API a runtime.
- Estrae comportamenti / euristiche.

3. Vengono sviluppate firme antivirus e comportamenti/euristiche.

Tecniche di evasioni dei malware

Gli scanner antivirus scoprono rapidamente i virus cercando attorno al punto di ingresso del programma.

Offuscamento del punto di ingresso (Entry Point Obfuscation):

- **Virus multicavity:** il virus prende il controllo più tardi, dopo che il programma è già stato avviato.
- Intercettazioni e sostituzione delle funzioni di libreria.

Tecniche di evasioni dei malware: offuscamento

Metamorfismo

- Genera varianti completamente nuove del codice a ogni infezione.
- Il codice appare diverso a livello di sintassi, ma mantiene la stessa semantica (stesso comportamento).
- Ancora più difficile da rilevare, anche con analisi euristiche o comportamentali.

Polimorfismo

- Cambia la struttura del codice (layout) a ogni infezione.
- Il payload rimane funzionalmente identico, ma viene crittografato o "impacchettato" con una chiave diversa a ogni infezione.
- Rende il rilevamento basato su firme poco efficace.
- Tuttavia, gli antivirus possono individuare la routine di decrittazione se non è offuscata.

Metamorfismo: codice originale

5B 00 00 00 00	pop ebx
8D 4B 42	lea ecx, [ebx + 42h]
51	push ecx
50	push eax
50	push eax
0F 01 4C 24 FE	sidt [esp - 02h]
5B	pop ebx
83 C3 1C	add ebx, 1Ch
FA	cli
8B 2B	mov ebp, [ebx]

**5B 00 00 00 00 8D 4B 42 51 50 50 0F 01 4C 24 FE 5B
83 C3 1C FA 8B 2B**

Metamorfismo: inserimento di codice “morto”

5B 00 00 00 00

8D 4B 42

51

50

90

50

40

0F 01 4C 24 FE

48

5B

83 C3 1C

FA

8B 2B

pop ebx

lea ecx, [ebx + 42h]

push ecx

push eax

nop

push eax

inc eax

sidt [esp - 02h]

dec eax

pop ebx

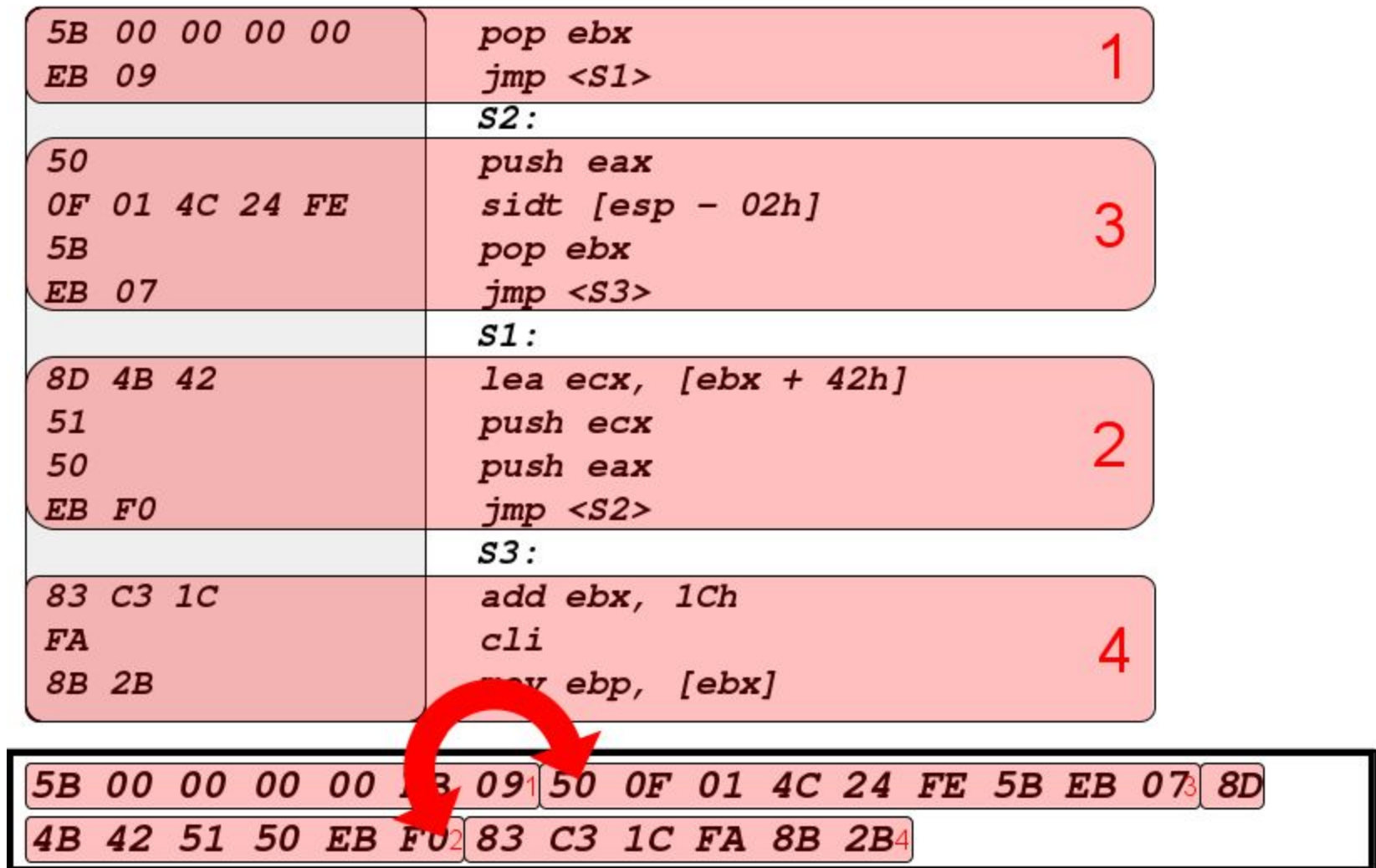
add ebx, 1Ch

cli

mov ebp, [ebx]

5B 00 00 00 00 8D 4B 42 51 50 90 50 40 0F 01 4C 24 FE
48 5B 83 C3 1C FA 8B 2B

Metamorfismo: riordino delle istruzioni



Altre tecniche impiegate dai malware

Periodo dormiente

Durante il quale non viene mostrato alcun comportamento malevolo.

Payload attivato da eventi

Spesso avviati da un canale di Command & Control (C&C).

Tecniche anti-virtualizzazione

Per evitare analisi in sandbox o macchine virtuali.

Cifratura / Packing

Simile al polimorfismo, ma con tecniche più avanzate usate in malware complessi.

Tecniche di rootkit

Per nascondere processi, file, attività e rendere difficile il rilevamento.

Malware analysis vs tecniche di evasione

Analisi statica

Pro:

- Visibilità completa del codice e copertura totale, inclusi codice dormiente o non raggiungibile.
- Sicura — nessun rischio di esecuzione del codice.

Contro:

- Le tecniche di offuscamento (es. metamorfismo, crittografia, packing) possono ostacolare l'analisi.
- Nessuna visibilità sul comportamento a runtime o sulla logica dipendente dall'ambiente.

Analisi dinamica

Pro:

- Supera l'offuscamento: si concentra sul comportamento reale.
- Rivela artefatti runtime (es. file generati, connessioni, modifiche al registro).

Contro:

- Copertura limitata del codice: i rami dormienti potrebbero non essere eseguiti.
- Rischio che il malware rilevi l'ambiente (anti-VM, anti-debugging).